

Programming Paradigms

- Object Oriented Programming is a programming paradigm
- An object has both state and behaviour
- OOP views the world as a network of interacting objects
- Every object has state and behaviour, along with interface and implementation
- Objects interact by sending messages
- Objects are an abstraction mechanism, providing encapsulation as well as packaging and information hiding
- A class contains instructions for creating specific kinds of objects
- Class level attributes are shared by all instances of a class
- Enumerations provide a fixed set of values that can be considered as a data type
- Connections between objects are known as associations, implemented with instance level variables
- Navigability tells us if an object taking part in an association knows about the other
- Navigability can be unidirectional or bidirectional
- Multiplicity dictates how many objects take part in each association
- A dependency is a need for one class to depend on another without having a direct association in the same direction
- Composition represents a whole-part relationship; when the whole is destroyed, the parts are also destroyed
- Aggregation represents a container-contained relationship, slightly weaker than composition
- Containee can exist even if container is deleted
- Association class represents additional information about an association
- Inheritance allows you to define a new class based on an existing class with the superclass being more general than the subclass
- Results in an is-a relationship, derived class is a subtype of the base class
- Multiple inheritance is not allowed in Java
- Method Overriding is when a subclass changes behaviour inherited from the parent class
- Method Overloading is when there are multiple methods with the same name but different type signatures
- An interface is a behaviour specification, resulting in an is-a relationship for the implementing class
- An abstract class is a representation of commonalities amongst subclasses and cannot be instantiated
- Abstract methods are methods without implementation
- Dynamic Binding is when method calls are resolved at runtime, as opposed to Static Binding which resolves it at compile time
- Overridden methods are resolved using dynamic binding, while Overloaded methods are resolved using static binding
- Polymorphism allows you to write code targeting superclasses to be used on subclasses
- Substitutability, Overriding, and Dynamic Binding combine to give polymorphism

Requirements

- A brownfield project replaces an existing product while a greenfield project is developed from scratch
- Stakeholders are individuals affected by the software project; they provide requirements
- Functional requirements specify what the system should do
- Non-Functional requirements specify the constraints under which the system is developed and operated

- NFRs are easier to miss and are sometimes critical to the success of the software
- Requirements can be prioritised based on the importance and urgency e.g. Essential, Typical, Novel
- Requirements that are out of scope can be discarded
- Requirements can be generated through methods like brainstorming, user surveys, observations, interviews, focus groups, prototyping, product surveys
- Prototypes are mock ups or scaled down versions of the product
- Prose is a textual description of a requirement
- Feature lists are a list of features of a product grouped according to some specified criteria
- User stories are short simple descriptions of a feature told from the perspective of the person who desires the new capability
- As a [user type] I can [function] so that [benefit]; benefit can be omitted if obvious
- More adjectives can be attached to the [user type] if needed
- User stories can be written at various levels
- High level user stories called epics cover bigger functionality and can be broken down to smaller user stories
- Condition of satisfaction can be added to user stories
- Other useful information includes priority, size, and urgency
- User stories aid scoping, estimation, and scheduling
- A use case is a description of a set of sequences or actions that a system performs to yield an observable result of value to an actor
- It describes an interaction between the user and the system for a specific functionality of the system
- Use cases capture functional requirements of a system and specified at various levels of detail
- Actors are roles played by a user
- Use cases only describe externally visible behaviour without the internal details
- A step gives intention of the user rather than mechanics
- The Main Success Scenario describes the most straightforward interaction for a given use case which assumes nothing goes wrong
- Extensions are add on to the MSS that describe alternative flow of events
- MSS should be self contained
- A use case can include another use case
- Preconditions specify the specific state the system is expected to be in before the use case starts
- Guarantees specify what the use case promises to give at the end of operations
- A glossary ensures that all stakeholders have a common understanding of noteworthy terms
- Supplementary requirements can be used to capture requirements that do not fit elsewhere

Design

- Product/External Design and Implementation/Internal design
- Abstraction is a technique for dealing with complexity by only considering details relevant to the current perspective or task at hand
- Data abstraction involves abstracting away lower level data items and thinking in terms of bigger entities
- Control abstraction involves abstracting away details of actual control flow to focus on tasks at a higher level
- Coupling is a measure of the degree of dependence between components or classes
- High coupling is discouraged as it is harder to maintain, integrate, and test
- X is coupled to Y if a change in Y could potentially require a change in X
- Cohesion is a measure of how strongly related and focused the various responsibilities of a component are
- Higher cohesion is better
- A model is a representation of something else; providing a simple view of a

complex entity because it only captures a selected aspect

UML Diagrams

- Models can help to analyse a complex entity related to software development, communicate information among stakeholders, and act as a blueprint for creating software
- Unified Modeling Language is a graphical notation to describe various aspects of a software system
- UML class diagrams describe the structure of an OOP solution
- There if no default visibility, it is simply unspecified
- Generic classes can be shown as well
- Associations are the main connections among the classes in a class diagram
- Associations are shown using a solid line, or can be shown as an attribute instead but not both
- Associations can have additional decorations such as labels or roles
- Multiplicity dictates how many objects take part in each association, no default multiplicity exists
- UML notes can add more information to a diagram
- Class diagrams can show different relationships between classes
- Association classes are denoted as connections to an association link using a dashed line
- Object diagrams show object structure at a given point of time, and can be used to complement class diagrams
- Conceptual class diagrams capture class structures in the problem domain
- Subset of class diagram notation without specific details
- Activity diagrams are used to model workflows
- Activity diagrams capture activities through actions and control flows that make them up
- Branch nodes show alternate paths with guard conditions
- Merge nodes show the end of alternate paths
- Fork nodes indicate the start of concurrent flows of control
- Join nodes indicate the end of parallel paths
- Rake indicates that part of activity is in a separate diagram
- Swimlane diagrams are partitioned activity diagrams which can show who is doing which action
- Sequence diagrams model the interaction between various entities in a system
- Optional elements may be omitted if there is no ambiguity
- Object deletion is shown with an X
- Alt frames indicate alternative paths
- Opt frames indicate optional paths
- Par frames indicate parallel paths
- Ref frame allows segment of interaction to be omitted and shown as a separate sequence diagram

Software Architecture

- Software architecture describes the overall organisation of the system and can be viewed as a very high level design
- Different architecture styles exist
- In the n-tier style, higher layers make use of service provided by lower layers
- The client-server style has at least one component playing the role of a server and at least one client component accessing the service of the server
- The transaction processing style divides the workload of the system down to a number of transactions which are then given to a dispatcher that controls the execution of each transaction
- The service-oriented architecture style builds applications by combining functionalities packaged as programmatically accessible services
- The event-drive style controls the flow of the application by detecting events from event emitters and communicating those events to interested event consumers

Software Design

- Design patterns are elegant reusable solutions to a commonly recurring problem
- Patterns are described by context, problem, solution, anti-patterns, consequences, and other useful information
- The singleton pattern dictates that certain classes should have no more than just one instance and use a factory together with class level fields
- The facade pattern says that access to other components should be done without exposing internal details, can be done with a middleman class
- The command pattern says that if a system needs to execute different tasks, we should use polymorphism for a general command object
- The Model View Controller pattern represents separating an application into 3 components; the view which the user sees, the controller that catches events and sends updates, and the model which stores and maintains data
- The observer pattern targets the problem where an object might be interested in being notified when changes occur to other objects, which can be implemented with an observer list and interface
- Top-Down design involves designing the higher-level design first before fleshing out lower levels, useful when design big systems where the higher level needs to be stable first
- Bottom-Up design involves designing the lower-level design first and putting them together later, not usually scalable, but might work when designing variations of existing systems of repurposing existing components
- Agile design can be contrasted with full upfront design

Implementation

- Integrated Development Environments help software engineers write code
- Debugging is the process of discovering defects in the program
- A guidelines is to maximise readability of code
- Avoid long methods and deep nesting or arrowhead style code
- Avoid complicated expressions, especially those having many negations and nested parentheses
- Avoid magic numbers or literals
- Make code as explicit as possible
- Lay out code so that it adheres to logical structure
- Do not trip up the reader
- Keep it simple, stupid
- Avoid premature optimisation
- Keep to the Single Level of Abstraction Principle
- Make the happy path prominent
- Follow a coding standard which aims to make entire codebase look like it was written by one person
- Name things well; use nouns for things and verbs for actions
- Avoid unsafe shortcuts
- Use the default branch
- Don't recycle variables or parameters
- Avoid empty catch blocks
- Delete dead code
- Minimise scope of variables and code duplication
- Comment minimally but sufficiently
- Do not repeat in comments; write comments targeting other programmers
- Refactoring is the process of restructuring code in small steps without modifying its external behaviour
- Refactoring is not rewriting or bug fixing
- Code smells are indications that usually correspond to deeper problems in the system
- Documentation can be for developer-as-user or developer-as-maintainer
- A top-down breadth-first approach is easier to understand than a bottom-up one
- Documentation should be accurate, comprehensive, and comprehensible
- Focus on providing higher level information that is not found in code or

- comments
- JavaDoc generates API documentation in HTML from comments in the source code
- Exceptions are used to deal with unusual but not entirely unexpected situations
- The exception handler catches a thrown exception and handles it
- Assertions are used to define assumptions about the program state so that the runtime can verify them
- Drastic action is taken when an assumption failure occurs
- Assertions can be disabled without modifying the code
- Logging is the deliberate recording of certain information during a program execution for future reference; useful for troubleshooting problems
- Defensive programming operates under the assumption that if you leave room for things to go wrong, they will go wrong
- Integration is the processes of combining parts of a software product to form a whole
- Late and one time integration involves waiting until all components are completed before integration
- Early and frequent involves integrating early and evolving each part in parallel in small steps
- Big band integration involves integrating all components at the same time
- Incremental integration involves integrating a few components at a time
- Build automation tools automate the steps of the build process usually by means of build scripts
- Continuous Integration integrates, builds, and tests automatically after each code change
- Continuous Deployment is when changes are not only integrated continuously, but also deployed to end-users at the same time
- Reusing tried and tested components can enhance the robustness of a new software system while reducing the manpower and time requirement although there might be some associated costs
- An Application Programming Interface specifies an interface through which other programs can interact
- A library is a collection of modular code that is general and can be used by other programs
- A framework is a reusable implementation of a software providing generic functionality that can be selectively customised to produce a specific application
- Libraries are meant to be used as is while frameworks are meant to be customised
- A platform provides a runtime environment for application

Quality Assurance

- Quality assurance is the process of ensuring that the software being built has the required levels of quality
- Quality assurance is made out of validation (checking if requirements are correct) and verification (checking if requirements are implemented correctly)
- Code reviews are the systematic examination of code with the intention of finding where the code can be improved
- Examples include pull request, pair programming, and formal inspections
- Static analysis is the analysis of code without actually executing it
- Formal verification uses mathematical techniques to prove the correctness of a program, can be used to prove the absence of errors
- Testing involves operating a system or component under specified conditions and observing or recording the results to make an evaluation of some aspect of the system or component
- Testing involves executing a set of test cases
- Test case failure is a mismatch between the expected behaviour and actual behaviour
- Testability is an indication of how easy it is to test a software under testing
- A regression is an unintended and undesirable effect created due to modification of a system

- Regression testing is re-testing software to detect regressions
- Developer testing is done by the developers themselves
- Delaying testing until the full product is complete has a number of disadvantages
- Unit testing involves testing individual units to ensure each piece works correctly
- Stubs help to isolate the software being tested from its dependencies
- Integration testing tests whether different parts of the software work together
- A hybrid of unit and integration testing is used
- System testing takes the whole system and tests it against the system specifications
- System test cases are based on the specified behaviour of the system and includes testing against the NFRs too
- Alpha testing is performed by the users under controlled conditions set by the software development team
- Beta testing is performed by a selected subset of target users of the system in their natural work setting
- Dogfooding is when creators use their own product
- Scripted testing involves writing a set of test cases based on expected behaviour then performing testing based on that set of test cases
- Exploratory testing devises test cases on the fly; creating new test cases based on results of the past test cases
- Scripted testing is more systematic and hence more likely to discover more bugs given sufficient time
- Acceptance testing involves testing the system to ensure it meets the user requirements
- Test automation involves running test cases and determining their result programmatically
- Input and output redirection can be used to semi-automate the testing of a CLI app
- A test driver is the code that drives the software being tested
- JUnit is a tool for automated testing of Java programs
- Moving as much logic as possible out of the GUI can make testing it easier
- Test coverage is a metric used to measure the extent to which testing exercises the code
- Examples include function / statement / decision / condition / path / entry / exit coverage
- Coverage can be measured using coverage analysis tools
- Dependency injection is the process of injecting objects to replace current dependencies with a different object
- Test Driven Development advocates writing the tests before writing the software while evolving functionality and tests in small increments
- Exhaustive testing is not practical because testing requires a massive number of test cases
- Every test case adds to the cost of testing
- Testing should be effective (finds a high percentage of existing bugs) and efficient (has a high rate of success)
- Each new test should target a potential fault that is not already targeted by existing test cases
- A positive test case is designed to produce expected behaviour
- A negative test case is designed to produce a behaviour that indicates an invalid situation
- Black box testing involves designing test cases based on the specified external behaviour
- White box testing involves designing test cases based on what is known about the software implementation
- Gray box is a mix of both
- Equivalence partitioning is a test case design technique that uses the above observation to improve the effectiveness and efficiency of testing
- An equivalence partition is a group of test inputs that are likely to be processed by the software the same way

- Avoid testing too many inputs from one partition and ensure all partitions are tested
- Equivalence partitions are usually derived from the specification of the software
- Boundary value analysis is a design heuristic based on the observation that bugs often result from incorrect handling of boundaries of equivalence partitions
- It suggest that values near boundaries are more likely to find bugs
- Typically you should choose one value from the boundary, one value just below the boundary, and one value just above the boundary
- A software can take multiple inputs, but testing all possible combinations across all inputs is effective but not efficient
- The all combinations strategy generates test cases for each unique combinations of test inputs
- The at least once strategy includes each test input at least once
- The all pairs strategy creates test cases so that for any given pair of inputs, all combinations between them are tested
- The random strategy generates test cases using one of the other strategies then picks a random subset
- Another heuristic is having each valid input at least once in a positive test case
- Another heuristic is handling invalid inputs individually before combining them
- We can mix multiple strategies together
- Use cases can be used for system and acceptance testing

Project Management

- Revision control is the process of managing multiple versions of a piece of information
- A revision is the state of a piece of information at a specific point in time resulting from changes made to it
- Revision Control Software are software tools that automate the process of revision control
- Git is the most widely used RCS today while Github is a web-based project hosting platform for projects using Git
- To put a folder under the control of Git, we initialise a repository in that folder
- We stage an untracked file to tell Git that we want it to be included in the next snapshot
- A modified a staged file it enters the modified state
- Staging applies regardless of whether a file is currently tracked
- Saving a snapshot is called committing and a saved snapshot is called a commit
- A timeline of commits is called a branch
- A Git data model consists of objects and refs
- A Git revision graph is a visualisation of a repository revision history
- Nodes represent commits while edges represents links between them
- Refs name and keep track of various points in a repository history
- A repository can be local or remote
- Content can be pushed from one repository to another
- We can omit files using a gitignore file
- A fork is a copy of a remote repository created on the same hosting service
- We can clone a repository to create a full copy of it locally
- To sync changes from a remote repository we need to fetch changes from remote and merge to apply the changes also known as pull
- Git lets you tag commits so they are easy to reference later
- We can use Git diff to compare changes between two points in the revision history
- Git stash allows to temporarily set aside uncommitted changes
- Git reset moves the tip of the current branch to a specific commit
- There are three types of resets: soft, mixed, hard
- Git branches allow you to develop multiple versions of work in parallel which can be merged together
- A fast forward merge moves the branch pointer to the tip of the other branch
- A squash merge takes all changes from one branch to combine into a single commit on the destination branch

- A merge conflict occurs when changes from two branches cannot be automatically combined
- Cherry picking allows us to synchronise branches by applying specific commits from one branch to another
- A centralised RCS uses a central remote repository that is shared
- A distributed RCS allows multiple remote or local repositories that work together
- In the forking workflow, the official code lives in a designated main repository while each developer works in their own fork and submits pull requests from separate branches back to the main repository
- In the feature branch workflow, everyone uses the same remote repository but on different branches
- The centralised workflow is similar but all changes are done in the master branch
- The Software Development Lifecycle describes stages of software development
- The sequential or waterfall model views software development as a linear process
- When one stage of the process is completed, it produces artifacts to be used in the next stage
- Works well for a project the produces software to solve a well understood problem but real world projects often tackle problems that are not well understood at the beginning
- The iterative model advocates producing the software by going through several iterations which produce a new version of the product
- In the breadth-first approach, an iteration evolves all major components and all functionality areas in parallel
- In the depth-first approach an iteration focuses on fleshing out only some components or some functionality area
- A project can be done as a mixture of both
- Agile models are another model of development
- Extreme Programming is an agile model which stresses customer satisfaction
- Scrum is a process skeleton that contains set of practice and predefined roles
- A scrum project is divided into iterations called sprints
- A work breakdown structure depicts information about tasks and their details in terms of subtasks
- The effort is traditionally measured in man hours and should have well defined tasks
- A milestone is the end of a stage which indicates significant progress
- A buffer is time set aside to absorb any unforeseen delays
- Issue trackers are commonly used to track task assignment and progress
- A Gantt chart is a 2D bar chart drawn as time vs tasks
- Different teams have different team structures
- Egoless teams are teams where every team member is equal in terms of responsibility and accountability
- In a chief programmer team, there is a single authoritative figure; the chief programmer
- In a strict hierarchy team, there is a strictly defined organisation among the team members

Principles

- Single Responsibility Principle says that a class should have exactly one reason to change
- Open Closed Principle says that a module should be open for extension but closed for modification
- Liskov Substitution Principle says that derived classes must be substitutable for their base classes
- SOLID is the above 3 together with Interface Segregation Principle and Dependency Inversion Principle
- Separation of Concerns says that to achieve better modularity, we should separate the code into distinct sections such that each section addresses a separate concern

- Law of Demeter says that an object should have limited knowledge of another object and it should only interact with objects that are closely related to it

